



Universidade do Minho

Escola de Engenharia

Programação Imperativa

LETI :: 2022/23

Outubro 2022

© António Esteves

© Slides adaptados de:

CSC230 - C e Software Tools, Computer Science Faculty, Duke University, USA

BE5B99CPL – C Programming Language, Jan Faigl, Czech Technical University in Prague

Bibliografia essencial:

C Programming: A Modern Approach, capítulos 5 e 6

3. Controlo do Fluxo de Execução: declarações de seleção e de ciclo

- Declarações
- Declarações de seleção
- Declarações de ciclo
- O operador condicional

- Declarações
- Declarações de seleção
- Declarações de ciclo
- O operador condicional

Declaração e Declaração composta (Bloco)

- Uma **declaração** (ou **instrução**) termina com um ponto e vírgula **;**
- Uma declaração contendo apenas um ponto e vírgula é uma declaração que não faz nada
- Um **bloco** é uma sequência de declarações
- Nas normas **ANSI C**, **C89** e **C90**: As **declarações de variáveis** têm que ser colocadas antes das declarações de outro tipo, mas esta restrição já não existe na norma **C99**
- O **início** e **fim** de um **bloco** é especificado por chavetas: **{** e **}**
- Um bloco pode estar dentro de outro bloco

```
void funcao1(void)
{ // início do bloco da função
  for (i = 0; i < 10; ++i) { // início do bloco do ciclo for
    ...
  } // fim do bloco do ciclo for
} // fim do bloco da função
```

➤ Declaração de **seleção**

- Declaração de seleção: `if` e `if-else`
- Declaração `switch`

➤ Declaração de **ciclo** (ou de **iteração**)

- `for`
- `while`
- `do-while`

➤ Declaração de **salto** - ramificação incondicional dum programa

- `continue`
- `break`
- `goto`
- ~~`return`~~

- Declarações
- Declarações de seleção
- Declarações de ciclo
- O operador condicional

Declaração de seleção: if e if-else (1)

- As declarações **if** e **if-else** são usadas para tomar decisões e executar blocos de código **condicionalmente** e **em alternativa**
- A sintaxe genérica da declaração **if-else** é a seguinte:

```
if( expressão )  
    bloco_verdadeiro  
else  
    bloco_falso
```

- Se a **expressão** for avaliada como **verdadeira** (diferente de zero), então executamos o código do **bloco_verdadeiro**
- Se a **expressão** for avaliada como **falsa** (zero), executamos o código do **bloco_falso** (se este estiver presente)

Declaração de seleção: **if-else** (2)

➤ **if** (expressão)
 instrução_V;
else

 instrução_F;

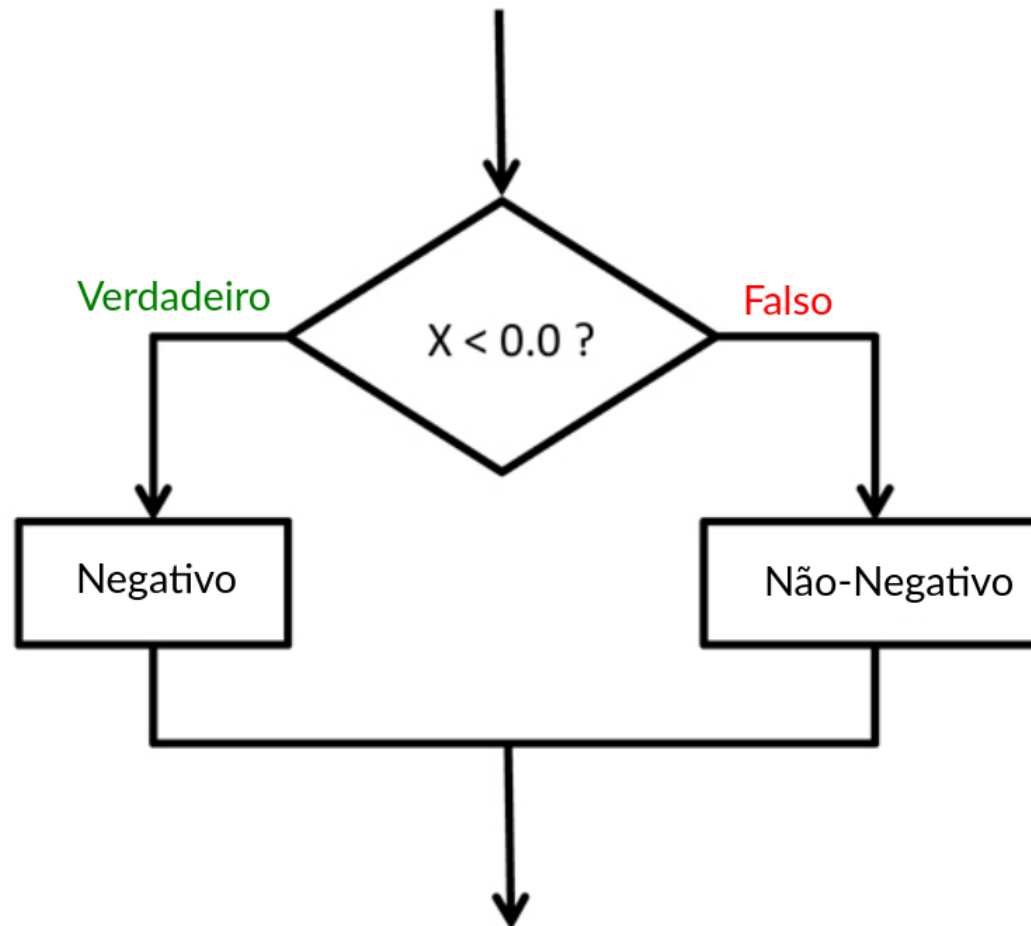
- A instrução_V e instrução_F podem ser compostas por várias instruções
- A seção **else** é **opcional**
- Declarações de seleção podem ser **aninhadas e encadeadas**

```
int max;
if (a > b) {
    if (a > c) {
        max = a;
    }
}
```

```
int max;
if (a > b) {
    ...
} else if (a < c) {
    ...
} else if (a == b) {
    ...
} else {
    ...
}
```


Declaração de seleção: **if-else** (3)

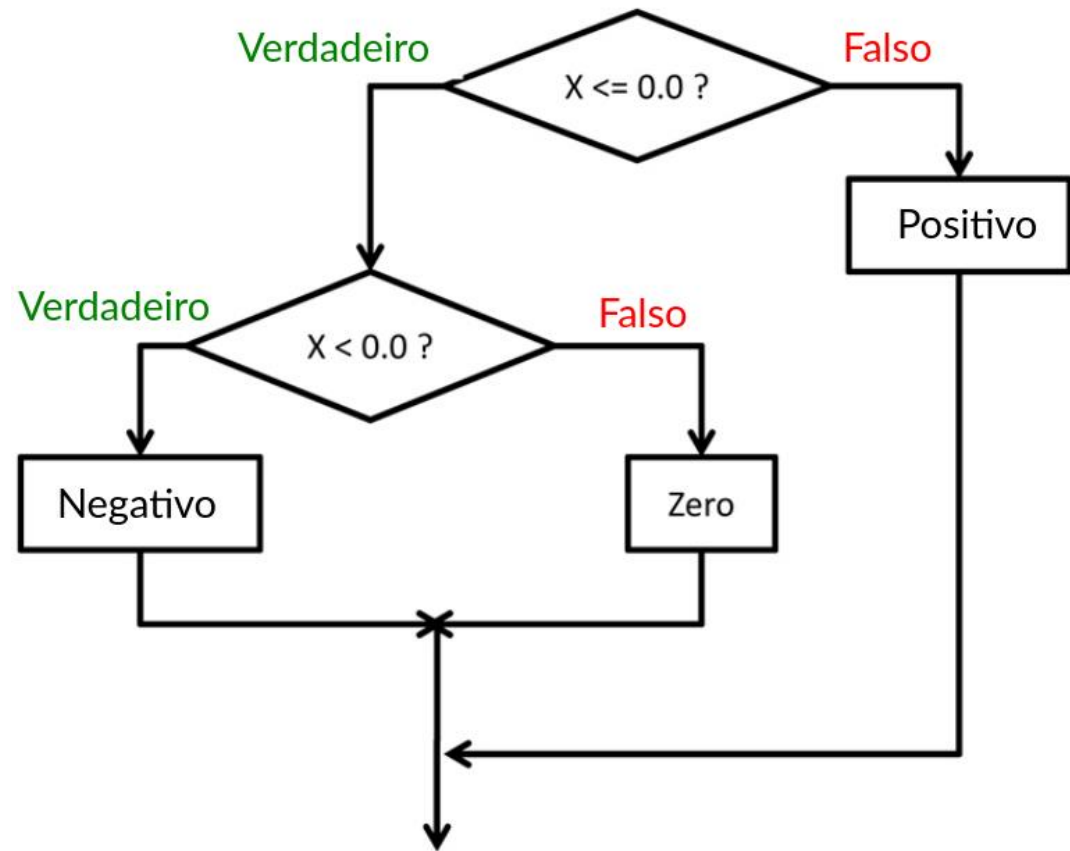
```
if( x < 0.0 )  
    printf( "x é negativo.\n" );  
else  
    printf( "x é não-negativo.\n" );
```



Declaração de seleção: if-else (4)

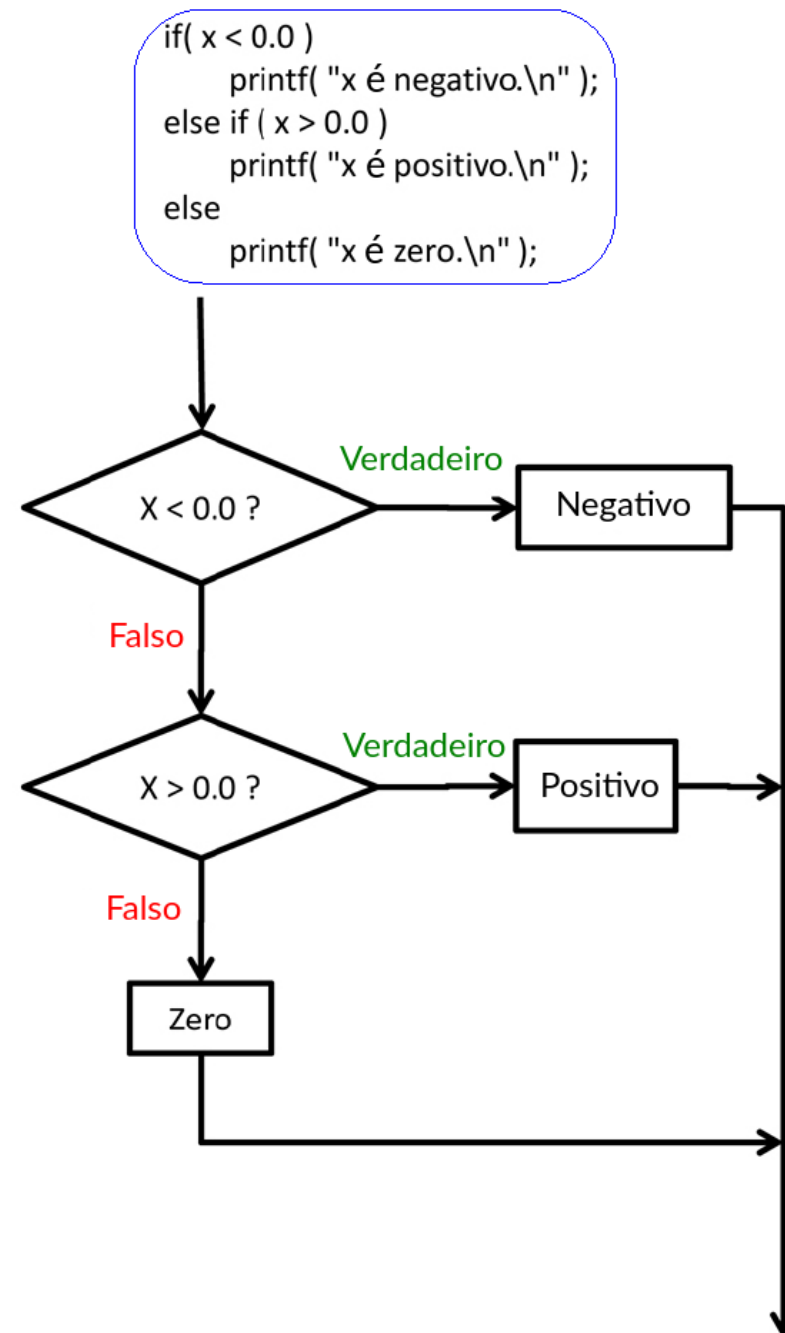
- Tanto o **bloco_verdadeiro** como o **bloco_falso** podem incluir qualquer tipo de código, incluindo outras declarações **if-else**
- Quando uma declaração **if-else** é utilizada no **bloco_verdadeiro** de outro **if-else**, criamos **if's aninhados**

```
if( x <= 0.0 ) {           // as chavetas só são obrigatórias se o
    if( x < 0.0 )          // if contiver várias instruções
        printf( "x é negativo.\n" );
    else
        printf( "x é zero.\n" );
} else
    printf( "x é positivo.\n" );
```



Declaração de seleção: if-else (5)

- Quando um **if-else** é incluído no **bloco_falso** de outro **if-else**, criamos **if's encadeados**



Declaração de seleção: Questão

Suponha que tem que indicar qual é a nota na época normal de PI (nota), para a qual contribuem 3 componentes: a nota dos testes (nT), a avaliação prática (nTP) e a participação nas aulas teóricas (partT). Estas componentes valem 60%, 30% e 10%, respetivamente. Para ter aprovação é preciso ter 7.5 nos testes e na prática. Como a avaliação prática não pode ser melhorada em exame, tendo abaixo de 7.5 o aluno não pode ir ao exame de recurso.

```
float nota, nT, nTP, partT;  
  
// nT=..., nTP=..., partT=...
```

```
//////////////////////////////// A  
nota = 0.6*nT + 0.3*nTP + 0.1*partT;  
if (nTP >= 7.5)  
{  
    if (nT >= 7.5)  
    {  
        if (nota >= 9.5)  
            printf("Aprovado com nota %f",nota);  
        else  
            printf("Reprovado com nota %f e admitido a exame",nota);  
    }  
    else  
        printf("Reprovado e admitido a exame");  
}  
else  
{  
    printf("Reprovado e não admitido a exame");  
}
```

```
//////////////////////////////// B  
nota = 0.6*nT + 0.3*nTP + 0.1*partT;  
if (nTP < 7.5)  
    printf("Reprovado e não admitido a exame");  
else if (nT < 7.5)  
    printf("Reprovado e admitido a exame");  
else if (nota < 9.5)  
    printf("Reprovado com nota %f e admitido a exame",nota);  
else  
    printf("Aprovado com nota %f",nota);
```

```
//////////////////////////////// C  
if (nTP < 7.5)  
    printf("Reprovado e não admitido a exame");  
if (nT < 7.5)  
    printf("Reprovado e admitido a exame");  
if (nota < 9.5)  
    printf("Reprovado com nota %f e admitido a exame",nota);  
else  
    printf("Aprovado com nota %f",nota);
```

Qual(ais) das alternativas seguintes implementa(m) corretamente a indicação da nota?

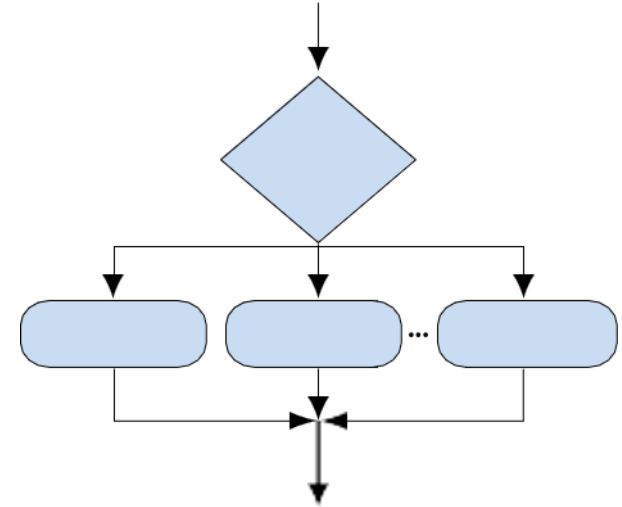
- Apenas A
- Apenas B
- Apenas C
- A e B
- A e B e C

A declaração **switch**

- Permite ramificar a execução dum programa numa de várias alternativas, escolhida de acordo com o valor de uma expressão que é avaliada num tipo enumerado inteiro: **char**, **short**, **int**, **enum**
- A sintaxe da declaração **switch** é:

```
switch (expressão) {
    case constante1:
        instruções1;
        break;
    case constante2:
        instruções2;
        break;
    . . .
    case constanten:
        instruçõesn;
        break;
    default:
        instruçõesdef;
        break;
}
```

onde as constantes são do mesmo tipo que a **expressão**



- Semântica da declaração **switch**:
 - Primeiro calcula-se o valor da **expressão**
 - Depois executam-se as instruções debaixo da etiqueta correspondente a esse valor
 - Se não houver nenhuma etiqueta com o valor da **expressão**, executam-se as **instruções_{def}** debaixo da palavra chave **default** (opcional)
- As declarações **switch** podem ser aninhadas

A declaração **switch**: exemplo

```
switch (v) {  
    case 'A':  
        printf("'A' maiusculo\n");  
        break;  
    case 'a':  
        printf("'a' minuscuro\n");  
        break;  
    default:  
        printf("Nao e' 'A' nem 'a'\n");  
        break;  
}
```

03-switch.c

Qual a versão mais estruturada?

```
if (v == 'A') {  
    printf("'A' maiusculo\n");  
}  
else if (v == 'a') {  
    printf("'a' minuscuro\n");  
}  
else {  
    printf(  
        "Não e' 'A' nem 'a'\n");  
}
```

O papel da declaração **break**

- A declaração **break** termina uma ramificação
- Se o **break** não for incluído, a execução dessa ramificação continua nas instruções da próxima alternativa **case**

Exemplo

```
int opcao = ...;
switch(opcao) {
    case 1:
        printf("Ramificação 1\n");
        break;
    case 2:
        printf("Ramificação 2\n");
    case 3:
        printf("Ramificação 3\n");
        break;
    case 4:
        printf("Ramificação 4\n");
        break;
    default:
        printf("Ramificação por omissão\n");
        break;
}
```

- se opcao == 1 executa
Ramificação 1
- se opcao == 2 executa
Ramificação 2
Ramificação 3
- se opcao == 3 executa
Ramificação 3
- se opcao == 4 executa
Ramificação 4
- se opcao == 5 executa
Ramificação default

03-demo-switch-break.c

Conteúdo

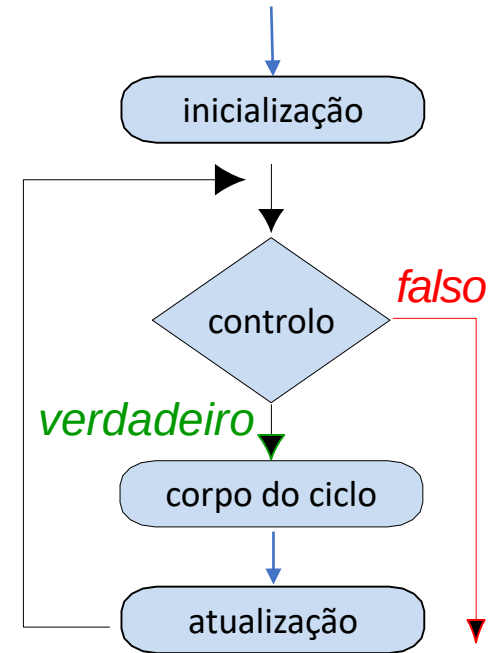
- Declarações
- Declarações de seleção
- Declarações de ciclo
- O operador condicional

Tarefas repetitivas

- Se lhe dissessem que o primeiro dia do mês de Março é numa quinta-feira e pedissem para escrever os pares (dia do mês, dia da semana) para a primeira semana desse mês, como resolveria o problema com o que aprendeu até aqui na UC?
- Se lhe dissessem que o primeiro dia do mês de Abril é numa quarta-feira e pedissem para escrever os pares (dia do mês, dia da semana) para todo o mês, como resolveria o problema com o que aprendeu até aqui na UC? Antevê outra forma mais elegante/viável de resolver o problema?
- E se lhe dissessem que o primeiro dia do ano é numa terça-feira e pedissem para escrever os tripletos (mês, dia do mês, dia da semana) para todo o ano, conseguiria resolver o problema com o que aprendeu na UC até aqui? Antevê uma forma viável de resolver o problema?

Ciclos **for** (1)

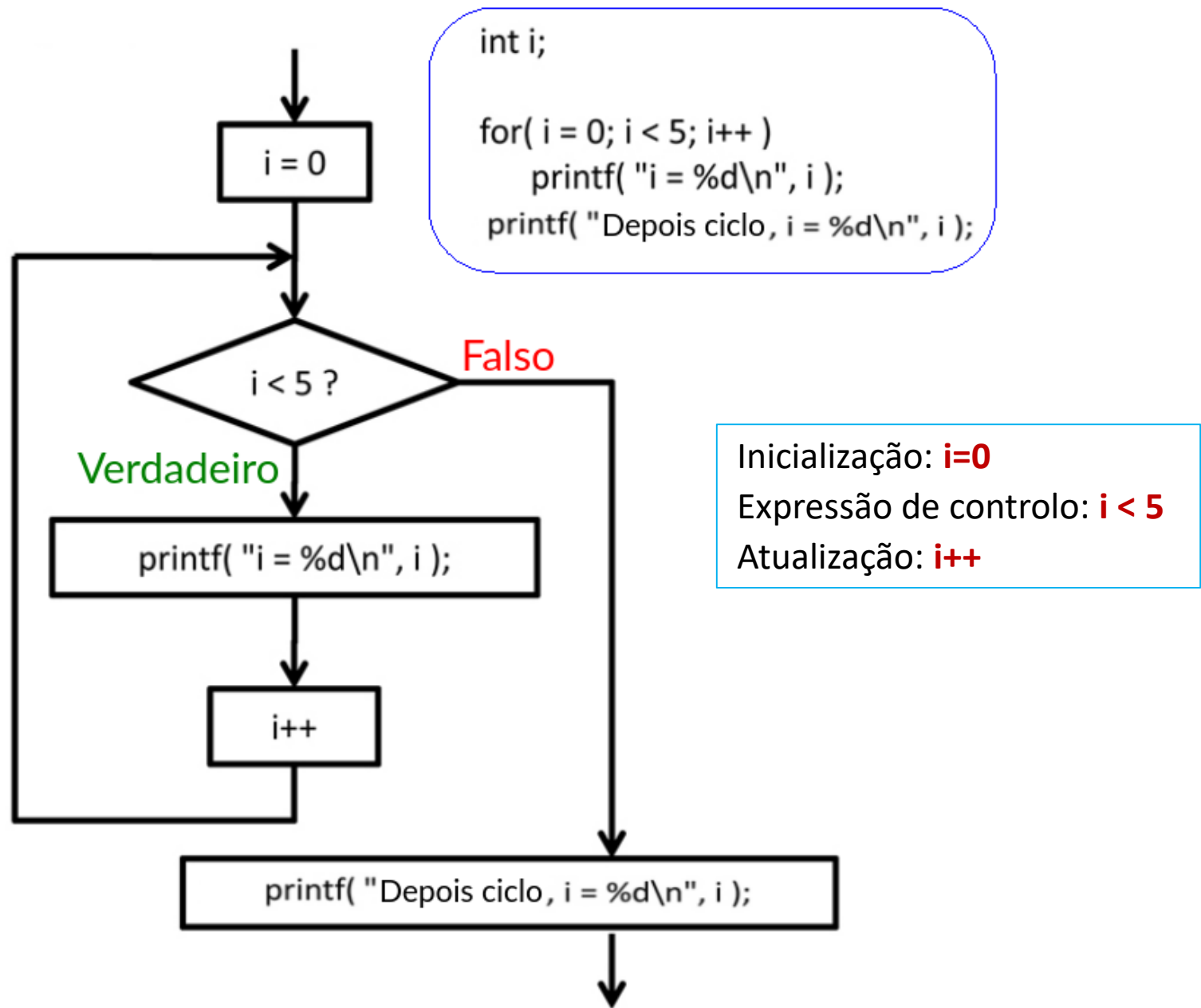
- Para indicar que se pretende repetir **N** vezes a execução de um bloco de instruções, sem ter que replicar esse bloco **N** vezes no programa, existem os ciclos → **while**, **do-while**, **for**
- Em quase todas as situações, os ciclos **for** e **while** incluem uma expressão de controlo que é avaliada antes de se executar o corpo do ciclo
- A declaração do ciclo **for** pode incluir 3 partes:
 - a **inicialização** – executada em primeiro lugar e apenas na primeira iteração do ciclo
 - a expressão de **controlo** – executada a seguir à inicialização e antes de iniciar cada nova **iteração** do ciclo
 - a expressão de **atualização** – executada em cada iteração após terminar a execução do corpo do ciclo



Ciclos **for** (2)

- O formato básico é: **for** ($expr_1$; $expr_2$; $expr_3$) *instrução*;
- Todas as $expr_i$ são expressões e são utilizadas para:
 - $expr_1$ – inicializar a(s) variável(eis) de controlo.
 - $expr_2$ – avaliar a expressão de controlo; se $expr_2$ for avaliada como verdadeira a *instrução* é executada, senão o ciclo termina.
 - $expr_3$ – atualizar a(s) variável(eis) de controlo.
- Qualquer das 3 expressões $expr_{1-3}$ pode ser omitida
- declaração **break** – se incluída dentro do corpo do ciclo, ao ser executada o **ciclo termina**
- declaração **continue** – se incluída dentro do corpo do ciclo, ao ser executada a **iteração atual** do ciclo **termina**
 - ➔ a execução **continua na** instrução de **atualização** $expr_3$
- Podemos escrever um ciclo infinito se omitirmos as 3 expressões:
for (;;) { ... }

➤ Exemplo:



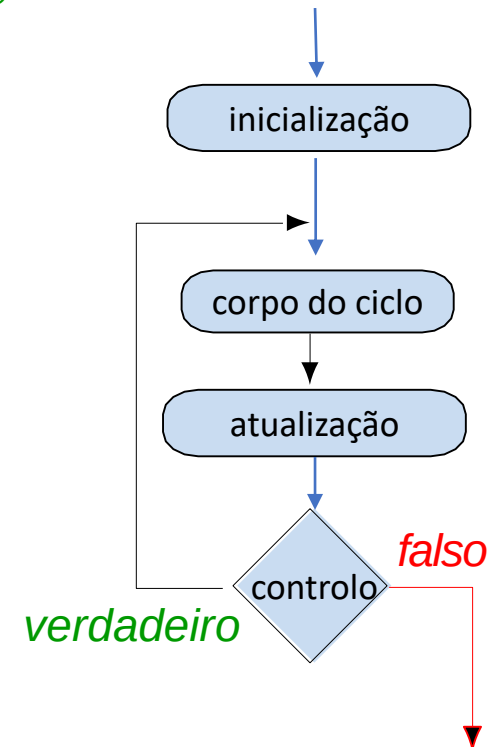
- A declaração do ciclo **while** apenas contempla a expressão de **controle**
- A(s) variável(eis) utilizadas na expressão de controle têm que ser declaradas e inicializadas fora do ciclo e atualizadas dentro do corpo do ciclo

```
int i = 0;           // inicialização fora do ciclo
while (i < 5) {      // controle do ciclo
    ... corpo do ciclo ...
    i++;             // atualização faz parte do corpo do ciclo
}
```

Ciclos **do-while**

- Nos ciclos **do-while** a expressão de controlo é avaliada depois de executar o corpo do ciclo

```
int i = 0;           // inicialização fora do ciclo
do {
    ... corpo do ciclo ...
    i++;             // atualização faz parte do corpo do ciclo
} while (i < 5);    // controlo do ciclo
```



A declaração **continue**

- A declaração **continue** pode ser utilizada no corpo de ciclos `for`, `while` e `do-while`

- Exemplos:

```
int i;
for (i = 0; i < 20; ++i)
{
    if (i % 2 == 0) {
        continue;
    }
    printf("%d\n", i);
}
```

03-continue.c

```
for (int i = 0; i < 9; ++i)
{
    printf("i:%d ", i);
    if (i % 3 != 2) {
        continue;
    }
    printf("\n");
}
```

03-demo-continue.c

```
gcc 03-demo-continue.c -o prog2
./prog2
```

```
i:0 i:1 i:2
i:3 i:4 i:5
i:6 i:7 i:8
```

Que escreve este programa no terminal?

A declaração **break** força o ciclo a terminar a execução

- O programa continua na primeira instrução a seguir ao ciclo
- Exemplo num ciclo **while**:

```
int i = 10;
while (i>0) {
    if (i == 5) {
        printf("i atingiu o valor 5, sair do ciclo\n");
        break;
    }
    i--;
    printf("Apos uma iteracao do ciclo i=%d\n", i);
}
```

03-break.c

- Exemplo num ciclo **for**:

```
for (int i = 0; i < 10; ++i) {
    printf("i:%d ", i);
    if (i > 5) {
        break;
    }
    if (i % 3 != 2) {
        continue;
    }
    printf("\n");
}
```

03-demo-break.c

```
gcc 03-demo-break.c -o demob
./demob

i:0 i:1 i:2
i:3 i:4 i:5
i:6
```

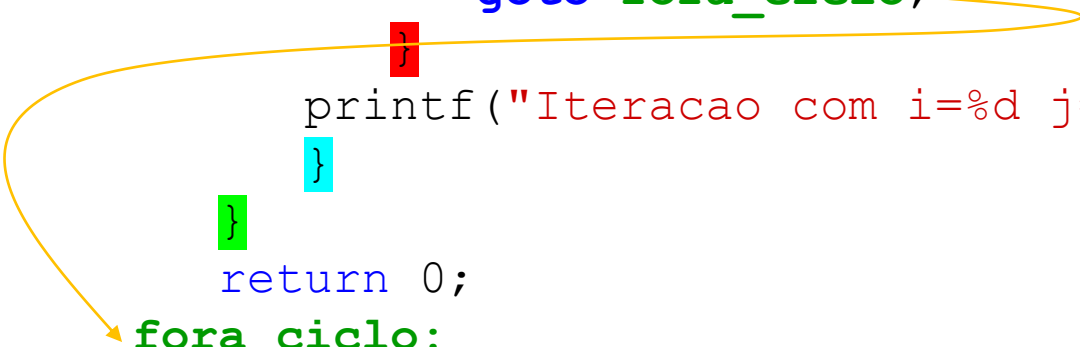

A declaração **goto**

- NOTA: não se recomenda a utilização de **goto** a não ser em casos excepcionais, como por exemplo para tratar situações de erro grave na execução do programa
- Transfere a execução para a instrução que se segue à etiqueta indicada
- Sintaxe: **goto** etiqueta;
- Só pode ser utilizado **dentro de uma função**
- A etiqueta alvo do salto tem que estar **fora do bloco** onde se inclui o **goto** mas dentro da mesma função

```

int limite = 3;
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 5; ++j) {
        if (j == limite) {
            goto fora_ciclo;
        }
        printf("Iteracao com i=%d j=%d\n", i, j);
    }
}
return 0;
fora_ciclo:
printf("Depois do ciclo\n");
return -1;

```



03-goto.c

- Uma declaração **break** dentro de **ciclos aninhados** termina o **ciclo mais interior** de entre aqueles que o **envolvem**

```
for (int i = 0; i < 3; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 1) {  
            break;  
        }  
    }  
}
```

- O ciclo mais exterior pode ser terminado com uma instrução goto:

```
for (int i = 0; i < 5; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 2) {  
            goto fora;  
        }  
    }  
}  
  
fora:
```

03-demo-goto.c

- Uma declaração **break** dentro de ciclos aninhados termina o ciclo mais interior

```
for (int i = 0; i < 3; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 1) {  
            break;  
        }  
    }  
}
```

i-j: 0-0
i-j: 0-1
i-j: 1-0
i-j: 1-1
i-j: 2-0
i-j: 2-1

- O ciclo mais exterior pode ser terminado com uma instrução goto:

```
for (int i = 0; i < 5; ++i) {  
    for (int j = 0; j < 3; ++i) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 2) {  
            goto fora;  
        }  
    }  
}  
  
fora:
```

03-demo-goto.c

- Uma declaração **break** dentro de ciclos aninhados termina o ciclo mais interior

```
for (int i = 0; i < 3; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 1) {  
            break;  
        }  
    }  
}
```

i-j: 0-0
i-j: 0-1
i-j: 1-0
i-j: 1-1
i-j: 2-0
i-j: 2-1

- O ciclo mais exterior pode ser terminado com uma instrução **goto**:

```
for (int i = 0; i < 5; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 2) {  
            goto fora;  
        }  
    }  
}
```

fora:

03-demo-goto.c

- Uma declaração **break** dentro de ciclos aninhados termina o ciclo mais interior

```
for (int i = 0; i < 3; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 1) {  
            break;  
        }  
    }  
}
```

i-j: 0-0
i-j: 0-1
i-j: 1-0
i-j: 1-1
i-j: 2-0
i-j: 2-1

- O ciclo mais exterior pode ser terminado com uma instrução **goto**:

```
for (int i = 0; i < 5; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        printf("i-j: %i-%i\n", i, j);  
        if (j == 2) {  
            goto fora;  
        }  
    }  
}
```

i-j: 0-0
i-j: 0-1
i-j: 0-2

fora:

03-demo-goto.c

- Qual(ais) o(s) ciclo(s) terminado(s) pela instrução **break**?

```
b = 0;
for ( i = 0; i < 10; i++ ) {
    for ( j = 0; j < 5; j++ ) {
        if (a[i][j] > 100)
            break;
        b += a[j];
    }
    printf("b = %d\n", b);
}
```

Exemplo: numeroPrimo() (1)

```
#include <stdbool.h>
#include <math.h>

bool numeroPrimo(int n)
{
    bool ret = true;
    for (int i = 2; i <= (int)sqrt((double)n); ++i) {
        if (n % i == 0) {
            ret = false;
            break;
        }
    }
    return ret;
}
```

03-demo-primos.c

- Assim que se encontrar um número **i** pelo qual **n** é divisível com resto zero, usamos um **break** para terminar o ciclo
- Não é necessário testar mais números pois o **n** já não pode ser primo
 - O programa executa em menos tempo

Exemplo: numeroPrimo() (2)

- O valor de `(int) sqrt((double) n)` não muda em todo o ciclo:

```
for (int i = 2; i <= (int) sqrt((double) n); ++i) { ... }
```
- Logo, podemos **calcular este valor uma única vez** e retirar o seu cálculo de dentro do ciclo

- Introduzimos a variável `maxValor` para guardar o valor:

```
for (int i = 2, maxValor = (int) sqrt((double) n);  
    i <= maxValor; ++i) { ... }
```

- Podemos ainda declarar `maxValor` como **constante**:

```
bool ret = true;  
  
const int maxValor = (int) sqrt((double) n);  
  
for (int i = 2; i <= maxValor ; ++i) { ... }
```

- Compilar e executar o exemplo `03-demo-primos.c`

```
gcc 03-demo-primos.c -lm -o primo  
./primo 13
```


- Declarações
- Declarações de seleção
- Declarações de ciclo
- **O operador condicional**

Operador condicional ?:

(1)

- Uma forma compacta de escrever uma declaração **if-else**

```
if (a > b)
    c = d;
else
    c = e;
```

consiste em utilizando o operador condicional **?:**

```
c = (a > b) ? d : e;
```

- O operador ternário **?:** é conhecido como **operador condicional**
- O operador condicional é equivalente a uma declaração **if-else**
- Mas como é um operador tem a vantagem de poder (i) ser usado numa **expressão** maior e (ii) **atribuir-se** o seu resultado a uma variável
- A sintaxe do operador condicional é:

expressão ? bloco_verdadeiro : bloco_falso
- Primeiro avalia-se a **expressão** e aplicam-se os efeitos colaterais
- Depois executa-se o código do **bloco_verdadeiro** ou do **bloco_falso** e aplicam-se os efeitos colaterais, dependendo de a **expressão** ter sido avaliada como **verdadeira** ou **falsa**

➤ Exemplos

```
max = i > j ? i : j;
```

```
abs = x >= 0 ? x : -x;
```

```
printf( "X é %s.\n", ( x < 0 ? "negativo" : "não negativo" ) );
```

```
salario = base + ( vendas > limite ? 1.5 : 1.0 ) * bonus;
```

Os parênteses são necessários no 4º exemplo porque os operadores **+** e ***** têm maior precedência do que **>** e **?:**

Expressões condicionais: exemplo Maior Divisor Comum (1)

```
int MaiorDivisorComum(int x, int y)
{
    int d;
    if (x < y) {
        d = x;
    } else {
        d = y;
    }
    while ( (x % d != 0) || (y % d != 0) ) {
        d = d - 1;
    }
    return d;
}
```

➤ A mesma funcionalidade com uma expressão condicional: `expr ? instrV : instrF`

```
int MaiorDivisorComum(int x, int y)
{
    int d = x < y ? x : y;
    while ( (x % d != 0) || (y % d != 0) ) {
        d = d - 1;
    }
    return d;
}
```

03-demo-mdc.c

Expressões condicionais: exemplo Maior Divisor Comum (2)

```
int MaiorDivisorComum(int x, int y)
{
    int d;
    if (x < y) {
        d = x;
    } else {
        d = y;
    }
    while ( (x % d != 0) || (y % d != 0) ) {
        d = d - 1;
    }
    return d;
}
```

➤ A mesma funcionalidade com uma expressão condicional: **expr ? instr_V : instr_F**

```
int MaiorDivisorComum(int x, int y)
{
    int d = x < y ? x : y;
    while ( (x % d != 0) || (y % d != 0) ) {
        d = d - 1;
    }
    return d;
}
```

03-demo-mdc.c

Exercícios sobre controlo de fluxo

1. Quais os valores de **d** e **g** após executar o código seguinte?

```
int d=11, g=12;
int e=13, f=14;
int h=15;
int a = 2, b = 3;
int x = 40, y = 30;

if (a < b) {
    d = e;
    if (x > y)
        g = h;
}
else
    d = f;
```

2. Escreva uma declaração **switch** equivalente ao código em baixo.

```
unsigned int a;
if ((a > 1) && (a <= 3))
    printf("calcular\n");
else if (a == 5)
    printf("adiar\n");
else if (a < 7)
    ;
else
    printf("invalido\n");
```

3. Se o primeiro dia do mês de Novembro ocorrer numa segunda-feira, utilize um ciclo **for** e um **switch** para escrever todos os pares (dia do mês, dia da semana) deste mês.